

How to Deploy the Northwind API

wiki/engineering/deployment-guide · Updated 2022-03-14 · Owner: Platform, SRE

Overview

This guide describes how to deploy the Northwind API to production. It is intended for engineers on the Platform and SRE teams who are responsible for shipping changes to the customer-facing service. Read it end to end before your first deploy, and keep it open the first few times you run a release.

The Northwind API is the backbone of our order-management platform. A failed or partial deploy can take down checkout for every customer, so the process below is deliberately conservative: it favours predictability over speed. When in doubt, slow down.

Audience and scope

This document covers deploying the Northwind API service only. It does not cover database schema migrations, infrastructure changes, or front-end releases, each of which has its own runbook. If your change touches more than the API service, coordinate with the owning teams before you begin.

Prerequisites

Before you can deploy, make sure your workstation is set up correctly.

- Install the Heroku CLI and log in with your corporate account. Deploys will not work

without a valid Heroku session.

- Request "deployer" access to the Northwind pipeline from your team lead. Access is granted

per person and reviewed quarterly.

- Clone the repository and confirm you can build it locally with the standard build task.
- Make sure you are on the main branch with a clean working tree. Never deploy from a

feature branch.

- Confirm you have access to the production dashboards so you can watch the rollout.

Access and permissions

Deployer access is tied to your corporate identity and your team membership. If you change teams, your access is re-evaluated. Do not share credentials or run a deploy on behalf of someone who lacks access; the audit trail must reflect who actually shipped the change.

Pre-deploy checklist

Run through this list before every release. Skipping steps here is the most common cause of failed deploys.

- The change has merged to main and the pull request is approved.
- All automated checks are green on the merge commit.
- You have read the change and understand its blast radius.
- There is no change freeze in effect for the current window.
- You have told the team you are about to deploy.

Deploying a release

To deploy the Northwind API, run `./deploy.sh` from the repo root. This script builds the app and pushes it directly to our Heroku dynos. Make sure you have the Heroku CLI installed and are logged in. Deployments typically take 3-4 minutes. If a deploy fails, re-run the script; it is idempotent.

The script performs the following steps in order:

- Verifies that you are authenticated and that the working tree is clean.
- Builds a production bundle and runs the smoke test suite.
- Tags the release with the current timestamp and short commit hash.
- Pushes the build to the Heroku dynos and waits for the health check to pass.

- Prints the release URL and the rollback command for the previous release.

Watching the rollout

Once the push completes, watch the dyno logs for at least five minutes.

Look for a clean boot, a successful database migration, and steady request latency. If error rates climb or the health check flaps, roll back immediately rather than trying to debug in production.

Deploy windows

Prefer to deploy during business hours when the team is around to help if something goes wrong. Avoid deploying late in the day on a Friday or immediately before a holiday. High-risk changes should be scheduled and announced in advance.

Worked example

A typical release looks like this. You merge an approved pull request to main in the morning. You announce in the team channel that you are deploying. You run the deploy script from a clean checkout of main. The build runs the smoke tests, tags the release, and pushes to the dynos. You watch the logs for five minutes, confirm the version endpoint reports your commit, check the dashboards, and post a short note with the release tag. The whole process takes under fifteen minutes including the watch window.

Rollback

Every deploy prints the command needed to roll back to the previous release. Keep that command handy during the rollout window. Rolling back restores the previous dyno slug and is usually faster than rolling forward with a fix. After a rollback, open an incident so the root cause is tracked and the bad release is not accidentally re-shipped.

When to roll back

Roll back if error rates rise, latency degrades, the health check will not stay green, or a migration fails. Do not try to debug a bad release live in production. Restore the previous release first, then investigate with the pressure off.

Post-deploy checklist

- Confirm the version endpoint reports the commit you just shipped.
- Check the dashboards for error rate, latency, and saturation.
- Post a short note in the team channel with the release tag and anything to watch.
- Close the deploy ticket and link the release notes.
- Update any status page entry if the change affects customers.

Troubleshooting

If the build step fails, the problem is almost always a dirty working tree or a failing smoke test; fix those locally and re-run. If the push to the dynos times out, check the platform status page before retrying. If the health check never

passes, roll back and investigate with a fresh pair of eyes. When in doubt, prefer rolling back over leaving a degraded release in production.

Common mistakes

- Deploying from a feature branch instead of main.
- Skipping the watch window and walking away immediately.
- Ignoring a flapping health check and hoping it settles.
- Forgetting to announce the deploy, so nobody can help if it goes wrong.

Frequently asked questions

How long should a deploy take? Three to four minutes for the push, plus a five-minute watch window. If it is taking much longer, something is wrong.

Can I deploy two changes at once? Prefer one change per deploy so that if something breaks you know exactly what caused it.

What if I am not sure the change is safe? Ask for a second opinion before you deploy. It is always cheaper to delay than to recover from a bad release.

Roles and responsibilities

The engineer running the deploy owns the rollout and the decision to roll back. The Platform team owns the deploy tooling and the pipeline. The SRE team owns the production environment and is the escalation point if a deploy causes an incident.

Related documents

See the CI/CD Pipeline Standards for how changes are built and promoted, the On-Call Runbook for what to do if a deploy causes an incident, and the Incident Severity Guide for how to classify customer impact.